

UNIVERSIDADE PAULISTA

WENDEL RICHARD DE CARVALHO – T214417

**AS TÉCNICAS CRIPTOGRÁFICAS, CONCEITOS, USOS E APLICAÇÕES
TÉCNICA ESCOLHIDA: RSA (RIVEST–SHAMIR–ADLEMAN)**

Trabalho apresentado como requisito parcial para obtenção de nota nas Atividades Práticas Supervisionadas (APS) do curso de Sistemas de Informação, disciplina vinculada: Introdução à Programação Estruturada (IPE).

SÃO PAULO

2026

UNIVERSIDADE PAULISTA

WENDEL RICHARD DE CARVALHO – T214417

**AS TÉCNICAS CRIPTOGRÁFICAS, CONCEITOS, USOS E APLICAÇÕES
TÉCNICA ESCOLHIDA: RSA (RIVEST–SHAMIR–ADLEMAN)**

Trabalho apresentado como requisito parcial
para obtenção de nota nas Atividades Práticas
Supervisionadas (APS) do curso de Sistemas
de Informação, 1º/2º Semestre — UNIP.
Disciplina vinculada: Introdução à Programação Estruturada (IPE).

SÃO PAULO

2026

SUMÁRIO

1	OBJETIVO DO TRABALHO	05
2	INTRODUÇÃO.....	06
3	CRİPTOGRAFIA — CONCEITOS GERAIS	08
3.1	Definição e Terminologia Básica	08
3.2	Criptografia Simétrica	09
3.3	Criptografia Assimétrica	09
3.4	Funções de Hash e Assinatura Digital	09
4	TÉCNICAS CRIPTOGRÁFICAS MAIS UTILIZADAS	10
4.1	Cifra de César e Vigenère	10
4.2	DES e Triple DES	10
4.3	AES — Advanced Encryption Standard	11
4.4	Diffie-Hellman e ElGamal.....	11
4.5	RSA — Visão Geral	12
5	DISSERTAÇÃO — TÉCNICA ESCOLHIDA: RSA.....	13
5.1	Estruturação, Conceitos e Fundamentação.....	13
5.2	Benefícios em Relação às Técnicas Anteriores	14
5.3	Aplicações e Sistemas que Utilizam RSA.....	15
5.4	Discussão Comparativa	16
5.5	Vulnerabilidades e Falhas.....	17
5.6	Melhorias Propostas e Implementadas.....	18
6	PROJETO DO PROGRAMA	19
6.1	Visão Geral da Arquitetura	19
6.2	Módulos e Funções Principais	20

6.3	Fluxo de Execução	20
6.4	Decisões de Projeto.....	21
7	RELATÓRIO COM AS LINHAS DE CÓDIGO	21
8	INTERDISCIPLINARIDADE E REFLEXÃO	24
9	BIBLIOGRAFIA	26
10	VIDEO DE APRESENTAÇÃO	27
11	FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS	28

1 OBJETIVO DO TRABALHO

O presente trabalho tem como objetivo analisar, compreender e aplicar a técnica criptográfica RSA (Rivest-Shamir-Adleman) em um contexto prático de segurança da informação. Para tanto, desenvolve-se uma aplicação funcional capaz de cifrar e decifrar mensagens de até 128 caracteres, demonstrando os fundamentos matemáticos do algoritmo e suas implicações reais no campo da segurança digital.

O cenário proposto envolve a necessidade de comunicação segura entre autoridades brasileiras e inspetores atuando em um navio apreendido pela Guarda Costeira Nacional, a 50 quilômetros da costa. O navio transportava resíduos tóxicos radioativos provenientes da Ásia e encontra-se em área de isolamento sanitário, sendo o acesso físico extremamente restrito. Todo contato é realizado por meio de helicópteros, e as comunicações eletrônicas devem garantir confidencialidade, autenticidade e integridade das mensagens trocadas. O sistema RSA implementado endereça diretamente essas exigências.

Do ponto de vista acadêmico, o trabalho busca consolidar os conteúdos estudados na disciplina de Introdução à Programação Estruturada (IPE), especialmente no que diz respeito ao desenvolvimento de algoritmos, manipulação de variáveis, estruturas de controle, funções e entrada e saída de dados. Adicionalmente, explora conceitos interdisciplinares de matemática discreta, teoria dos números e segurança computacional.

Em termos específicos, o grupo visa: (a) apresentar a fundamentação matemática do RSA, compreendendo a aritmética modular e o problema da fatoração de inteiros; (b) contextualizar historicamente a criptografia assimétrica e seu impacto na segurança da internet; (c) implementar o algoritmo em linguagem Python sem dependências externas; (d) desenvolver uma interface gráfica interativa para demonstração em tempo real durante a apresentação; e (e) elaborar uma análise crítica das vulnerabilidades, limitações e perspectivas futuras do RSA no contexto da computação quântica.

2 INTRODUÇÃO

Desde os primórdios da civilização, a necessidade de proteger informações sensíveis levou o ser humano a desenvolver métodos para ocultar o conteúdo de mensagens. Os primeiros sistemas criptográficos, como a Cifra de César na Roma Antiga e o Atbash hebraico, baseavam-se em substituições simples de caracteres. Embora eficazes para sua época, tais métodos eram vulneráveis a técnicas elementares de análise de frequência, descobertas já no século IX pelo matemático árabe Al-Kindi em sua obra *Manuscrito sobre a Decifragem de Mensagens Criptografadas*.

A criptografia moderna evoluiu de forma dramática ao longo do século XX, impulsionada pelas guerras mundiais e, posteriormente, pela revolução digital. A máquina alemã Enigma, amplamente utilizada na Segunda Guerra Mundial para cifrar comunicações militares, demonstrou tanto o potencial quanto as limitações dos sistemas eletromecânicos de cifragem. Sua decifração pelos aliados, liderada por Alan Turing e sua equipe em Bletchley Park, lançou as bases para a criptoanálise computacional moderna e acelerou o desenvolvimento dos primeiros computadores eletrônicos.

Um marco fundamental na criptografia contemporânea ocorreu em 1976, quando Whitfield Diffie e Martin Hellman publicaram o revolucionário artigo *New Directions in Cryptography*, introduzindo o conceito de criptografia assimétrica, também denominada criptografia de chave pública. Nesse paradigma, cada participante possui um par de chaves matematicamente relacionadas: uma chave pública, que pode ser amplamente divulgada, e uma chave privada, que deve ser mantida em absoluto sigilo. Uma mensagem cifrada com a chave pública somente pode ser decifrada por quem possui a chave privada correspondente.

Um ano depois, em 1977, os pesquisadores do Massachusetts Institute of Technology (MIT) Ronald Rivest, Adi Shamir e Leonard Adleman concretizaram essa ideia no algoritmo RSA, que se tornaria a pedra angular da segurança digital por décadas. O RSA fundamenta-se na dificuldade computacional do problema da fatoração de inteiros: dado um número n composto por dois primos muito grandes p e q , é computacionalmente inviável determinar p e q a partir de n em tempo polinomial, com os algoritmos clássicos conhecidos atualmente.

Hoje, o RSA é empregado em praticamente todos os sistemas que exigem transmissão segura de dados pela internet: o protocolo HTTPS que protege transações bancárias online, sistemas de autenticação SSH em servidores, certificados digitais da ICP-Brasil utilizados em documentos eletrônicos, e o processo inicial de troca de chaves em protocolos como o TLS 1.3. Sua relevância transcende o domínio puramente técnico, impactando diretamente a privacidade, a economia e a soberania digital dos Estados nacionais.

No contexto deste trabalho, a aplicação do RSA mostra-se particularmente pertinente ao cenário de isolamento do navio contaminado: inspetores autorizados possuem chaves privadas exclusivas armazenadas em dispositivos seguros, enquanto a chave pública pode ser transmitida livremente para qualquer helicóptero ou posto de comando costeiro. Somente o inspetor legítimo, portador da chave privada, poderá decifrar as instruções de acesso às áreas de risco radiológico, garantindo que informações críticas não sejam interceptadas por agentes não autorizados.

A disciplina de Introdução à Programação Estruturada fornece os alicerces práticos necessários à implementação: compreensão de algoritmos, uso de funções, estruturas de repetição e condicionais, e manipulação de tipos de dados numéricos. A implementação do RSA em Python sem bibliotecas criptográficas externas constitui, portanto, tanto um exercício avançado de programação estruturada quanto uma demonstração da elegância matemática subjacente ao algoritmo.

3 CRIPTOGRAFIA — CONCEITOS GERAIS

3.1 DEFINIÇÃO E TERMINOLOGIA BÁSICA

Criptografia é a ciência e arte de transformar informações legíveis (texto plano ou plaintext) em uma representação ilegível (texto cifrado ou ciphertext), de modo que somente partes autorizadas possam reverter essa transformação e recuperar a informação original. O processo de transformação é denominado cifragem ou encriptação, enquanto o processo inverso recebe o nome de decifragem ou deciptação.

Os elementos fundamentais de qualquer sistema criptográfico são: (a) o algoritmo de cifragem, que define o método matemático de transformação; (b) a chave, parâmetro que controla o comportamento do algoritmo; (c) o texto plano, que constitui a mensagem original; e (d) o texto cifrado, que é a saída do processo de cifragem. A segurança de um sistema criptográfico moderno deve residir exclusivamente no sigilo da chave, e não no sigilo do algoritmo — princípio enunciado por Auguste Kerckhoffs em 1883 e amplamente adotado na criptografia contemporânea como Princípio de Kerckhoffs.

A criptoanálise é a área que estuda métodos para quebrar sistemas criptográficos sem o conhecimento da chave. Entre as técnicas clássicas de criptoanálise destacam-se a análise de frequência, que explora a distribuição estatística de letras em idiomas naturais; os ataques de texto plano conhecido, em que o analista possui pares de texto plano e cifrado; e os ataques de força bruta, que testam exaustivamente todas as chaves possíveis.

3.2 CRIPTOGRAFIA SIMÉTRICA

Na criptografia simétrica, a mesma chave é utilizada tanto para cifrar quanto para decifrar a mensagem. Esse modelo apresenta alta eficiência computacional, sendo adequado para cifragem de grandes volumes de dados. Contudo, impõe um problema crítico denominado distribuição de chaves: como transmitir de forma segura a chave compartilhada entre as partes comunicantes, especialmente em canais públicos e potencialmente inseguros?

Exemplos notáveis de algoritmos simétricos incluem o DES (Data Encryption Standard), publicado pelo NIST em 1977 e hoje considerado inseguro devido ao tamanho reduzido de sua chave de 56 bits; o 3DES, variante de maior robustez que aplica o DES três vezes; e o AES (Advanced Encryption Standard), adotado como

padrão em 2001 e amplamente utilizado até os dias atuais, com chaves de 128, 192 ou 256 bits. Os algoritmos simétricos subdividem-se em cifras de bloco, que operam sobre blocos fixos de bits, e cifras de fluxo, que operam sobre cada bit individualmente.

3.3 CRIPTOGRAFIA ASSIMÉTRICA

A criptografia assimétrica, ou de chave pública, resolve o problema da distribuição de chaves ao empregar dois valores matematicamente relacionados: a chave pública e a chave privada. A chave pública pode ser divulgada abertamente sem comprometer a segurança do sistema, pois é computacionalmente inviável derivar a chave privada a partir dela — dada a dificuldade de certos problemas matemáticos subjacentes, como a fatoração de inteiros (RSA) ou o logaritmo discreto (ElGamal, ECDSA).

O custo dessa segurança adicional é o desempenho: operações assimétricas são ordens de magnitude mais lentas do que as simétricas equivalentes. Por isso, na prática, sistemas modernos como o TLS utilizam criptografia assimétrica apenas para a troca inicial de uma chave de sessão simétrica, que então é empregada para cifrar o tráfego efetivo. Essa abordagem híbrida combina a segurança do modelo assimétrico com a eficiência do simétrico.

3.4 FUNÇÕES DE HASH E ASSINATURA DIGITAL

Funções criptográficas de hash transformam uma entrada de tamanho arbitrário em uma saída de tamanho fixo (digest), com as propriedades de resistência a colisões e irreversibilidade. Algoritmos como SHA-256 e SHA-3 são amplamente utilizados para verificação de integridade de arquivos, armazenamento seguro de senhas e composição de assinaturas digitais.

As assinaturas digitais combinam criptografia assimétrica com funções de hash para prover autenticidade, integridade e não-repúdio. No esquema RSA, o remetente cifra o hash da mensagem com sua chave privada; o destinatário verifica a assinatura calculando novamente o hash da mensagem recebida e comparando com o resultado de decifrar a assinatura com a chave pública do remetente. Esse mecanismo é a base legal dos documentos eletrônicos no Brasil, regulamentados pela Medida Provisória 2.200-2/2001.

4 TÉCNICAS CRIPTOGRÁFICAS MAIS UTILIZADAS E CONHECIDAS

4.1 CIFRA DE CÉSAR E VIGENÈRE

A Cifra de César, criada por Júlio César no século I a.C., consiste em substituir cada letra do alfabeto pela letra que ocupa uma posição k posições à frente. Com $k = 3$, a letra A torna-se D, B torna-se E, e assim por diante. Apesar de sua simplicidade histórica, esse método é trivialmente quebrável por força bruta, já que há apenas 25 possibilidades de chave no alfabeto latino, ou por análise de frequência, que compara a distribuição de letras no texto cifrado com a distribuição conhecida do idioma original.

A Cifra de Vigenère, desenvolvida no século XVI e atribuída ao diplomata francês Blaise de Vigenère, estende a substituição de César ao utilizar uma chave de múltiplos caracteres, aplicando um deslocamento diferente para cada posição da mensagem. Por séculos foi considerada inquebrável, recebendo o título de *le chiffre indéchiffrable*. Contudo, o método foi completamente quebrado no século XIX pelo matemático Charles Babbage e, independentemente, pelo oficial prussiano Friedrich Kasiski, mediante análise do índice de coincidência e do teste de Kasiski para descoberta do tamanho da chave.

4.2 DES E TRIPLE DES

O Data Encryption Standard (DES), desenvolvido pela IBM e adotado como padrão americano pelo National Institute of Standards and Technology (NIST) em 1977, foi por décadas o algoritmo simétrico dominante no mundo corporativo e financeiro. Opera sobre blocos de 64 bits com chave de 56 bits, aplicando 16 rodadas de uma função de Feistel. Em 1999, o projeto Distributed.net realizou um ataque de força bruta bem-sucedido em menos de 24 horas utilizando hardware especializado, demonstrando definitivamente a insuficiência do tamanho de chave de 56 bits para os padrões computacionais modernos.

O Triple DES (3DES) surgiu como solução paliativa de transição, aplicando o algoritmo DES três vezes consecutivas com até três chaves distintas, elevando a segurança efetiva para 112 bits no modo EDE (Encrypt-Decrypt-Encrypt) com duas chaves. Apesar de ainda em uso em sistemas legados bancários e de pagamento, o 3DES foi oficialmente depreciado pelo NIST em 2019 e está sendo progressivamente substituído pelo AES em todos os novos sistemas.

4.3 AES — ADVANCED ENCRYPTION STANDARD

O AES foi selecionado pelo NIST em 2001 após um rigoroso processo de avaliação pública internacional que durou cinco anos, no qual quinze algoritmos candidatos foram submetidos à análise da comunidade criptográfica mundial. O algoritmo Rijndael, criado pelos criptógrafos belgas Joan Daemen e Vincent Rijmen, saiu vencedor. Opera sobre blocos de 128 bits com chaves de 128, 192 ou 256 bits, aplicando respectivamente 10, 12 ou 14 rodadas de quatro transformações: SubBytes (substituição não-linear), ShiftRows (permutação de linhas), MixColumns (mistura de colunas) e AddRoundKey (adição da subchave da rodada).

O AES é atualmente o padrão simétrico mais utilizado no mundo, presente em VPNs, criptografia de disco (BitLocker no Windows e FileVault no macOS), aplicativos de mensagens (WhatsApp e Signal), redes Wi-Fi (WPA2 e WPA3) e praticamente todos os sistemas de telecomunicações modernos. Hardware dedicado para operações AES está integrado em processadores Intel desde 2010 (extensão AES-NI) e em chips Apple Silicon, permitindo cifragem em velocidades superiores a 10 Gbps com consumo de energia mínimo.

4.4 DIFFIE-HELLMAN E ELGAMAL

O protocolo de troca de chaves Diffie-Hellman, publicado em 1976 por Whitfield Diffie e Martin Hellman, permite que duas partes estabeleçam um segredo compartilhado em um canal público inseguro sem jamais transmitir o segredo em si. O método baseia-se na dificuldade computacional do problema do logaritmo discreto: dados g (gerador), p (primo grande) e o valor g elevado a a módulo p , é computacionalmente inviável determinar o expoente a . O protocolo está na base dos mecanismos de troca de chaves no TLS, SSH e IPsec.

O algoritmo ElGamal, proposto por Taher Elgamal em 1985, estende o conceito de Diffie-Hellman para cifragem assimétrica completa, utilizando o mesmo problema do logaritmo discreto. Possui variantes em curvas elípticas, denominadas ECDH (para troca de chaves) e ECDSA (para assinaturas digitais), que oferecem segurança equivalente ao RSA com chaves significativamente menores. Uma chave ECDSA de 256 bits proporciona segurança comparável a uma chave RSA de 3072 bits, sendo preferida em ambientes com recursos computacionais limitados como dispositivos móveis e IoT.

4.5 RSA — VISÃO GERAL

O RSA, objeto central deste trabalho, é detalhado na dissertação da seção seguinte. Destaca-se neste contexto comparativo que é o algoritmo assimétrico de maior adoção histórica e ainda amplamente utilizado em infraestruturas de chave pública (PKI), certificados digitais X.509 e protocolos de autenticação. Sua popularidade deve-se à maturidade de implementações, à extensa análise de segurança acumulada em mais de quatro décadas, e à ampla disponibilidade de hardware e software otimizados para suas operações.

5 DISSERTAÇÃO — TÉCNICA ESCOLHIDA: RSA

5.1 ESTRUTURAÇÃO, CONCEITOS E FUNDAMENTAÇÃO

O RSA, desenvolvido em 1977 pelos pesquisadores do MIT Ronald Rivest, Adi Shamir e Leonard Adleman, constitui o primeiro algoritmo prático de criptografia assimétrica a ser publicado e amplamente adotado. Seu funcionamento baseia-se em dois problemas matemáticos considerados intratáveis computacionalmente com algoritmos clássicos: a fatoração de inteiros grandes e o problema RSA (calcular raízes modulares de alta ordem sem conhecer a fatoração do módulo). A segurança do algoritmo é diretamente proporcional ao tamanho do módulo n — quanto maior n , mais difícil sua fatoração.

5.1.1 Geração do Par de Chaves

O processo de geração de um par de chaves RSA segue os passos descritos a seguir. Primeiro, escolhem-se dois números primos grandes e distintos p e q , cada um com pelo menos 512 bits de comprimento. Segundo, calcula-se o módulo público $n = p \times q$ — o produto $p \times q$ forma o módulo que é incluído em ambas as chaves e pode ser compartilhado livremente, pois fatorá-lo de volta em p e q é computacionalmente inviável. Terceiro, calcula-se a função totiente de Euler: $\varphi(n) = (p - 1) \times (q - 1)$, que representa a quantidade de inteiros positivos menores que n coprimos com n e deve permanecer em segredo. Quarto, escolhe-se o expoente público e tal que $1 < e < \varphi(n)$ e $\text{mdc}(e, \varphi(n)) = 1$; na prática usa-se $e = 65537$, o número de Fermat $F_4 = 2$ elevado a 16 mais 1, que é primo e possui representação binária com poucos bits 1, tornando a exponenciação modular mais eficiente. Por fim, calcula-se o expoente privado d como o inverso modular de e módulo $\varphi(n)$: $d \times e \equiv 1 \pmod{\varphi(n)}$, obtido pelo Algoritmo de Euclides Estendido.

As chaves resultantes são: Chave Pública $P_k = (e, n)$, que pode ser publicada livremente, e Chave Privada $S_k = (d, n)$, que deve ser mantida em sigilo absoluto. Os valores p , q e $\varphi(n)$ são destruídos após a geração das chaves.

5.1.2 Cifragem e Decifragem

Para cifrar uma mensagem M representada como um inteiro no intervalo fechado $[0, n-1]$ com a chave pública (e, n) , calcula-se: $C = M$ elevado a e módulo n . Para decifrar o texto cifrado C com a chave privada (d, n) , calcula-se: $M = C$ elevado a d módulo n . A corretude desse processo é garantida pelo Teorema de Euler: para M e n coprimos, tem-se M elevado a $\varphi(n)$ congruente a 1 módulo n , e portanto M

elevado a $(e \times d)$ é congruente a M elevado a $(1+k \times \phi(n))$, que é congruente a M módulo n , para algum inteiro k . Na implementação deste trabalho, cada caractere é convertido para seu código ASCII (inteiro entre 0 e 127), cifrado individualmente e transmitido como número inteiro.

5.1.3 Exponenciação Binária Modular

A operação central do RSA, M elevado a e módulo n , envolve expoentes com milhares de bits. O algoritmo ingênuo, que realizaria e multiplicações, seria completamente impraticável. A exponenciação binária, também denominada square-and-multiply, reduz o número de operações para $O(\log e)$ multiplicações modulares, tornando a cifragem viável mesmo com expoentes muito grandes. O algoritmo percorre os bits do expoente da direita para a esquerda (ou da esquerda para a direita, em variantes alternativas), elevando ao quadrado o resultado a cada passo e multiplicando pelo resultado parcial apenas quando o bit corrente é 1.

5.2 BENEFÍCIOS EM RELAÇÃO ÀS TÉCNICAS ANTERIORES

O RSA representou um salto qualitativo em relação a todos os sistemas criptográficos disponíveis anteriormente a 1977. Sua principal contribuição foi resolver, de forma matematicamente elegante, o problema da distribuição de chaves, que havia sido a limitação fundamental da criptografia simétrica por séculos. Em sistemas anteriores como o DES ou as cifras clássicas, as partes comunicantes precisavam de um canal seguro preexistente para acordar a chave antes de iniciar a comunicação — um paradoxo fundamental: para se comunicar com segurança, era necessário já possuir um canal seguro.

Com o RSA, qualquer entidade pode publicar sua chave pública abertamente em um diretório público e qualquer remetente pode cifrar uma mensagem para ela sem qualquer contato prévio seguro. Isso foi revolucionário para a internet nascente, onde partes desconhecidas precisam trocar informações sigilosas constantemente, em escala global, sem oportunidade de estabelecimento prévio de segredos compartilhados.

Adicionalmente, o RSA viabilizou as assinaturas digitais, permitindo que documentos eletrônicos sejam autenticados com propriedades de não-repúdio — o assinante não pode negar posteriormente ter assinado o documento, pois somente ele possui a chave privada. Esse recurso é a base do comércio eletrônico, dos certificados digitais e da identidade digital na internet moderna. Tecnologias

anteriores simplesmente não ofereciam esse conjunto completo de propriedades: confidencialidade, autenticidade, integridade e não-repúdio em um único framework matemático coerente.

5.3 APLICAÇÕES E SISTEMAS QUE UTILIZAM RSA

Desde sua publicação, o RSA tornou-se onipresente nos sistemas de segurança digital. O protocolo HTTPS e TLS utilizam o RSA na fase de handshake para autenticar o servidor e estabelecer as chaves de sessão. O certificado X.509 do servidor contém sua chave pública RSA, assinada por uma Autoridade Certificadora confiável, e o cliente verifica essa assinatura para confirmar a identidade do servidor antes de qualquer troca de dados.

O SSH (Secure Shell) utiliza autenticação por chave pública RSA em servidores Linux e Unix. O servidor armazena a chave pública do usuário no arquivo `authorized_keys`; a autenticação ocorre por um protocolo de desafio-resposta em que o servidor cifra um nonce com a chave pública e o cliente demonstra posse da chave privada ao devolver o nonce decifrado, sem jamais transmiti-la pela rede.

A Infraestrutura de Chave Pública (PKI) e, especificamente, a ICP-Brasil utilizam RSA para emissão de certificados digitais. Certificados do tipo A3, utilizados para assinatura de declarações de imposto de renda, notas fiscais eletrônicas (NF-e) e documentos com validade jurídica, baseiam-se em RSA de 2048 bits armazenado em tokens criptográficos resistentes a adulteração. Os aplicativos de mensagens PGP e GPG utilizam RSA para cifrar a chave de sessão AES com que o corpo da mensagem é cifrado, e para assinar digitalmente as mensagens.

No cenário específico deste trabalho, o RSA seria aplicado da seguinte forma: o posto de comando costeiro geraria um par de chaves para cada inspetor autorizado. A chave pública de cada inspetor seria registrada no sistema de controle de acesso do navio. Mensagens de autorização de acesso a áreas restritas seriam cifradas com a chave pública do inspetor destinatário e transmitidas via rádio ou satélite pelo helicóptero. Somente o inspetor portador do token com a chave privada correspondente poderia decifrar e executar as instruções.

5.4 DISCUSSÃO COMPARATIVA

Comparado ao AES, o principal algoritmo simétrico em uso, o RSA apresenta a vantagem fundamental de resolver o problema da distribuição de chaves ao custo de ser ordens de magnitude mais lento para cifragem de grandes

volumes de dados. Operações RSA-2048 são tipicamente 1.000 vezes mais lentas que AES-128 para cifrar dados equivalentes. Por essa razão, o RSA raramente é usado para cifrar dados diretamente; em vez disso, cifra-se a chave de sessão AES com RSA e usa-se AES para os dados — a denominada criptografia híbrida, que combina as vantagens de ambos os paradigmas.

Em relação ao ECC (Elliptic Curve Cryptography), o RSA apresenta desvantagem crescente em termos de eficiência: uma chave RSA de 3.072 bits oferece segurança equivalente a uma chave ECC de apenas 256 bits. Com o crescimento de dispositivos IoT com recursos computacionais e de bateria limitados, o ECC tem ganhado espaço em cenários onde o RSA era tradicionalmente utilizado. O TLS 1.3, publicado em 2018, removeu suporte a vários cipher suites baseados em RSA para troca de chaves e passou a exigir ECDHE para garantia de forward secrecy.

Frente às cifras clássicas como César e Vigenère, o RSA apresenta superioridade matemática absoluta e incondicional: enquanto essas são quebráveis por análise de frequência em questão de segundos com qualquer computador moderno, a quebra do RSA-2048 exigiria, com o melhor algoritmo clássico conhecido (General Number Field Sieve), um esforço computacional estimado em aproximadamente 10 elevado a 30 operações básicas — inviável mesmo para clusters de supercomputadores operando pelo tempo de vida do universo.

5.5 VULNERABILIDADES E FALHAS

Apesar de sua solidez matemática, o RSA apresenta diversas vulnerabilidades que merecem atenção cuidadosa em qualquer implementação séria. Em relação ao tamanho das chaves, o RSA-512 foi fatorado em 1999 mediante esforço distribuído, e o RSA-768 foi fatorado em 2009 em projeto acadêmico. O NIST recomenda atualmente o mínimo de 2048 bits para uso até 2030, e 3072 bits para aplicações com horizonte temporal além disso.

Os ataques de tempo (timing attacks) exploram o fato de que implementações ingênuas do RSA consomem tempo variável dependendo dos valores dos bits da chave privada, permitindo que um adversário que mede o tempo de resposta infira bits da chave. Implementações seguras utilizam expoentes aleatorizados por uma técnica denominada blinding para equalizar o tempo de execução independentemente dos dados processados.

O ataque de Bleichenbacher (1998), conhecido como Million Message Attack, explorou o esquema de preenchimento PKCS#1 v1.5 para decifrar mensagens RSA mediante aproximadamente um milhão de consultas ao servidor. O atacante envia mensagens cuidadosamente construídas e observa se o servidor as aceita ou rejeita, usando as respostas para, iterativamente, estreitar o intervalo de valores possíveis para o texto plano. A solução é o esquema OAEP (Optimal Asymmetric Encryption Padding), definido no PKCS#1 v2.x, que adiciona aleatoriedade ao processo de cifragem.

A ameaça da computação quântica representa o risco de longo prazo mais significativo para o RSA. O algoritmo de Shor, proposto por Peter Shor em 1994, demonstra que um computador quântico com número suficiente de qubits lógicos poderia fatorar inteiros RSA em tempo polinomial. Embora computadores quânticos práticos ainda estejam distantes de atingir essa capacidade para módulos de 2048 bits, o NIST iniciou em 2016 um processo de padronização de algoritmos pós-quânticos, publicando os primeiros padrões finais em 2024: CRYSTALS-Kyber (para encapsulamento de chaves) e CRYSTALS-Dilithium (para assinaturas), ambos baseados em problemas de reticulados matemáticos.

5.6 MELHORIAS PROPOSTAS E IMPLEMENTADAS

Ao longo de décadas, diversas melhorias foram propostas e implementadas no ecossistema RSA. O RSA-OAEP (Optimal Asymmetric Encryption Padding) é o esquema de preenchimento probabilístico definido no PKCS#1 v2.1 que elimina as vulnerabilidades do preenchimento determinístico original. Cada operação de cifragem produz um texto cifrado diferente para a mesma mensagem, impossibilitando ataques que exploram a determinismo do esquema anterior. É o padrão obrigatório para cifragem RSA em todos os sistemas modernos.

O RSA-PSS (Probabilistic Signature Scheme) é o esquema de assinatura probabilístico com garantias de segurança demonstravelmente mais fortes que o determinístico PKCS#1 v1.5. O TLS 1.3 exige explicitamente o uso de RSA-PSS para assinaturas, proibindo o uso do esquema anterior. A aceleração por hardware, mediante extensões como Intel AES-NI e suporte nativo a multiplicação modular em CPUs modernas, reduz significativamente o overhead computacional do RSA.

A adoção de chaves efêmeras para garantia de forward secrecy representa uma melhoria arquitetural importante. Sistemas modernos utilizam RSA apenas para

autenticação das partes, empregando Diffie-Hellman efêmero (ECDHE) para a troca das chaves de sessão. Isso garante que, mesmo que a chave privada RSA seja comprometida no futuro, sessões anteriores permaneçam protegidas, pois as chaves efêmeras são descartadas após cada sessão e não podem ser reconstituídas.

6 PROJETO (ESTRUTURA) DO PROGRAMA

6.1 VISÃO GERAL DA ARQUITETURA

O programa foi desenvolvido integralmente em Python 3, utilizando exclusivamente a biblioteca tkinter, que integra a instalação padrão do Python e dispensa qualquer dependência externa. A interface gráfica é organizada em cinco abas temáticas, cada uma correspondendo a uma funcionalidade do sistema: cifragem, decifragem, geração e exibição de chaves, visualizador animado passo a passo e painel informativo sobre o RSA. A escolha do tkinter atende diretamente ao requisito do instrutivo de apresentação do programa em funcionamento em sala de aula, pois o arquivo pode ser executado em qualquer computador com Python instalado mediante um único comando.

A arquitetura do código separa claramente duas camadas: o núcleo matemático, composto por funções e pela classe RSA independentes de qualquer interface, e a camada de apresentação, implementada pela classe AplicacaoRSA que herda de tk.Tk. Essa separação garante que a lógica criptográfica possa ser testada isoladamente e que a interface possa ser substituída ou estendida sem alterar o algoritmo.

6.2 MÓDULOS E FUNÇÕES PRINCIPAIS

O núcleo matemático é composto por quatro funções auxiliares e uma classe principal. A função `potencia_modular` implementa a exponenciação binária modular (square-and-multiply), calculando base elevado a expoente módulo em $O(\log \text{expoente})$ multiplicações. A função `algoritmo_euclides_estendido` retorna o máximo divisor comum e os coeficientes de Bézout para dois inteiros. A função `inverso_modular` deriva o expoente privado d mediante o algoritmo de Euclides estendido. A função `millar_rabin` implementa o teste de primalidade probabilístico de Miller-Rabin com k rodadas, verificando se um inteiro candidato é primo com probabilidade de erro inferior a 4 elevado a menos k .

A classe RSA encapsula os parâmetros do par de chaves e os métodos de alto nível: `gerar_chaves(bits)` coordena a geração aleatória de p e q , o cálculo de n , $\phi(n)$, e e e d ; `cifrar(mensagem)` converte cada caractere para ASCII e aplica $C = M$ elevado a e módulo n ; `decifrar(cifrado)` aplica $M = C$ elevado a d módulo n e reconverte para caracteres.

A classe `AplicacaoRSA` herda de `tk.Tk` e constrói toda a interface gráfica. Operações potencialmente lentas, como a geração de primos grandes, são executadas em threads secundárias via `threading.Thread` para que a interface permaneça responsiva durante o processamento. O método `after()` do `tkinter` é utilizado para retornar ao thread principal e atualizar os widgets com os resultados.

6.3 FLUXO DE EXECUÇÃO

Ao iniciar o programa, a interface é construída e imediatamente disparada uma thread para gerar o par de chaves RSA inicial (256 bits por padrão), exibindo uma mensagem de status durante o processo. Após a geração, todos os campos de chave pública e privada nas abas Cifrar e Decifrar são preenchidos automaticamente.

O fluxo de cifragem na aba correspondente segue: (a) o usuário digita a mensagem de até 128 caracteres, com contador de caracteres em tempo real; (b) ao clicar em Cifrar, o método `_cifrar()` valida a entrada, chama `rsa.cifrar()` e exibe os inteiros resultantes na caixa de saída, junto com a representação hexadecimal; (c) o botão Enviar p/ Decifrar copia automaticamente o texto cifrado para a aba de decifragem e muda a aba ativa. O fluxo de decifragem é simétrico: o usuário cola os inteiros, clica em Decifrar e o método `_decifrar()` recupera e exibe a mensagem original com confirmação de integridade.

6.4 DECISÕES DE PROJETO

A cifragem caractere a caractere foi mantida por razões pedagógicas: cada operação $C = M \text{ elevado a } e \text{ módulo } n$ é visível individualmente no visualizador, facilitando a compreensão do algoritmo durante a apresentação. O visualizador animado usa primos pequenos fixos ($p = 61$, $q = 53$) para que os valores numéricos sejam legíveis na tela, enquanto as abas de cifragem e decifragem operam com as chaves reais de 256 ou 512 bits.

O esquema de cores escuro (dark theme) foi adotado para melhor legibilidade em projeção em sala de aula. Todas as constantes de estilo são centralizadas no início do arquivo, facilitando personalização. O programa não utiliza bibliotecas de terceiros, atendendo ao espírito da disciplina de Introdução à Programação Estruturada.

7 RELATÓRIO COM AS LINHAS DE CÓDIGO

7.1 NÚCLEO MATEMÁTICO — EXPONENCIAÇÃO E INVERSO

MODULAR

```
def potencia_modular(base, expoente, modulo):
    """Exponenciação binária:  $O(\log e)$  multiplicações."""
    if modulo == 1:
        return 0
    resultado = 1
    base = base % modulo
    while expoente > 0:
        if expoente % 2 == 1:                # bit = 1
            resultado = (resultado * base) % modulo
        expoente >>= 1                       # próximo bit
        base = (base * base) % modulo        # eleva ao quadrado
    return resultado

def algoritmo_euclides_estendido(a, b):
    if a == 0:
        return b, 0, 1
    mdc, x1, y1 = algoritmo_euclides_estendido(b % a, a)
    return mdc, y1 - (b // a) * x1, x1

def inverso_modular(a, m):
    """ $d = a^{-1} \bmod m$  via Euclides Estendido."""
    mdc, x, _ = algoritmo_euclides_estendido(a % m, m)
    if mdc != 1:
        raise ValueError("Inverso modular nao existe")
    return x % m
```

7.2 GERAÇÃO DE PRIMOS E PAR DE CHAVES

```
def miller_rabin(n, k=10):
    """Teste de primalidade probabilístico."""
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0: return False
    r, d = 0, n - 1
    while d % 2 == 0:                # escreve  $n-1 = 2^r * d$ 
        r += 1; d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
```

```

        x = potencia_modular(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = potencia_modular(x, 2, n)
            if x == n - 1: break
        else: return False
    return True

def gerar_primo(bits=256):
    while True:
        n = random.getrandbits(bits) | (1 << (bits-1)) | 1
        if miller_rabin(n): return n

def gerar_chaves(self, bits=256):
    self.p = gerar_primo(bits)      # primo p
    self.q = gerar_primo(bits)      # primo q (≠ p)
    while self.q == self.p:
        self.q = gerar_primo(bits)
    self.n = self.p * self.q
    self.phi_n = (self.p - 1) * (self.q - 1)
    self.e = 65537                  # expoente público (Fermat F4)
    self.d = inverso_modular(self.e, self.phi_n)
    self.chave_publica = (self.e, self.n)
    self.chave_privada = (self.d, self.n)

```

7.3 CIFRAGEM E DECIFRAGEM

```

def cifrar(self, mensagem):
    """C = M^e mod n para cada caractere."""
    if len(mensagem) > 128:
        raise ValueError("Maximo de 128 caracteres.")
    e, n = self.chave_publica
    cifrado = []
    for caractere in mensagem:
        m = ord(caractere)          # codigo ASCII
        c = potencia_modular(m, e, n) # C = M^e mod n
        cifrado.append(c)
    return cifrado

def decifrar(self, cifrado):
    """M = C^d mod n para cada valor cifrado."""
    d, n = self.chave_privada
    return "".join(

```

```

        chr(potencia_modular(c, d, n)) #  $M = C^d \bmod n$ 
    for c in cifrado
)

```

7.4 INTERFACE GRÁFICA — TRECHO PRINCIPAL

```

class AplicacaoRSA(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("RSA Criptografia - APS 2026")
        self.geometry("1000x720")
        self.configure(bg=COR_BG)
        self.rsa = RSA()
        self._build_header() # cabeçalho com nome e RA
        self._build_tabs()   # 5 abas temáticas
        self._build_status() # barra de status inferior
        # Gera chaves em thread para não travar a UI
        threading.Thread(target=self._gerar_chaves_auto,
                        daemon=True).start()

    def _cifrar(self):
        msg = self.txt_msg.get("1.0", "end-1c")
        cifrado = self.rsa.cifrar(msg)
        cifrado_str = " ".join(str(c) for c in cifrado)
        self._set_text_widget(self.txt_cifrado, cifrado_str)

    def _decifrar(self):
        entrada = self.txt_dec_input.get("1.0", "end-1c").strip()
        partes = [int(x) for x in entrada.split()]
        resultado = self.rsa.decifrar(partes)
        self._set_text_widget(self.txt_dec_output, resultado)

if __name__ == "__main__":
    app = AplicacaoRSA()
    app.mainloop()

```

8 INTERDISCIPLINARIDADE E REFLEXÃO SOBRE O APRENDIZADO

A elaboração deste trabalho evidenciou, de forma concreta, a natureza profundamente interdisciplinar da área de Sistemas de Informação. O desenvolvimento do sistema RSA mobilizou conhecimentos de ao menos quatro campos distintos do saber, cada um contribuindo de forma insubstituível para a totalidade do projeto. Essa integração transversal de saberes não foi um objetivo secundário do trabalho, mas uma consequência natural e inevitável da complexidade do objeto estudado.

Da matemática, foram fundamentais a Teoria dos Números — especialmente os conceitos de congruência modular, função totiente de Euler, primalidade e algoritmo de Euclides — e a Álgebra Abstrata, com os conceitos de grupo, anel e corpo sobre os quais a corretude do RSA é formalmente demonstrada pelo Teorema de Euler. Sem compreensão desses fundamentos, seria impossível implementar o algoritmo corretamente ou entender por que a decifragem recupera exatamente a mensagem original.

Da disciplina de Introdução à Programação Estruturada, os conceitos de funções recursivas, variáveis, estruturas de controle iterativas, manipulação de tipos numéricos de precisão arbitrária e decomposição de problemas em módulos reutilizáveis foram diretamente aplicados na implementação Python. A decomposição em funções coesas com responsabilidades bem definidas reflete os princípios de projeto de algoritmos estudados em sala de aula durante o semestre.

Da Segurança da Informação, a análise das vulnerabilidades do RSA — ataques de timing, padding oracle attacks de Bleichenbacher, ameaça quântica do algoritmo de Shor — demonstrou que a segurança real vai muito além da correção matemática do algoritmo. Aspectos de engenharia de software, gestão segura de chaves, modelagem de ameaças e defesa em profundidade são igualmente críticos para que um sistema criptográfico seja seguro na prática.

O cenário aplicado ao navio contaminado introduziu, ainda, elementos de ética profissional e responsabilidade social: a criptografia não é um fim em si mesma, mas um instrumento a serviço da proteção de vidas, da privacidade e da soberania. Inspectores que acessam áreas de risco radiológico dependem da confiabilidade dos sistemas de autenticação; falhas criptográficas nesse contexto têm consequências que transcendem o domínio digital e afetam diretamente a segurança humana. Essa

perspectiva enriqueceu a compreensão do grupo sobre o impacto social da computação.

A vivência deste projeto consolidou, portanto, não apenas competências técnicas específicas em algoritmos criptográficos e programação, mas uma visão sistêmica e integradora do papel do profissional de computação na sociedade contemporânea. O RSA, surgido de uma pesquisa matemática abstrata no MIT em 1977, tornou-se a espinha dorsal da segurança digital global — exemplificando como o conhecimento teórico rigoroso, aliado à habilidade de implementação prática, pode transformar profundamente a realidade.

9 BIBLIOGRAFIA

- INSTITUTO NACIONAL DE TECNOLOGIA DA INFORMAÇÃO (ITI). **Certificação Digital — Perguntas Frequentes**. Brasília: ITI, 2025. Disponível em: <https://www.gov.br/iti/pt-br/acesso-a-informacao/perguntas-frequentes/certificacao-digital>. Acesso em: 23 jun. 2026.
- NUCLEO DE INFORMAÇÃO E COORDENAÇÃO DO PONTO BR (NIC.br); CERT.br. **Cartilha de Segurança para Internet**. São Paulo: Comitê Gestor da Internet no Brasil, 2024. Disponível em: <https://cartilha.cert.br/livro/cartilha-seguranca-internet.pdf>. Acesso em: 23 jun. 2026.
- RECEITA FEDERAL DO BRASIL. **Leiaute dos Certificados Digitais da RFB (e-CPF, e-CNPJ)**. Brasília: Receita Federal, 2021. Disponível em: <https://www.receita.fazenda.gov.br/acrsf/LeiautedeCertificadosdaSRF.pdf>. Acesso em: 23 jun. 2026.
- BORGES, Daniele Helena Bonfim. **Criptografia RSA**. 2017. Dissertação (Mestrado Profissional em Matemática) — Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2017. Disponível em: https://teses.usp.br/teses/disponiveis/55/55136/tde-06042017-164507/publico/DanieleHelenaBonfim_revisada.pdf. Acesso em: 23 jun. 2026.
- UNIVERSIDADE DE SÃO PAULO (USP). **e-Aulas USP: Segurança da Informação — Algoritmos e Protocolos**. São Paulo: USP. Disponível em: <https://eaulas.usp.br/portal/course.action?course=18210>. Acesso em: 23 jun. 2026.
- BANCO CENTRAL DO BRASIL (BCB). **Manual de Segurança do Pix**. Versão 3.6. Brasília: BCB, 2024. Disponível em: https://www.bcb.gov.br/content/estabilidadefinanceira/cedsfn/Manual_de_Seguranca_PIX.pdf. Acesso em: 23 jun. 2026.
- INSTITUTO DE MATEMÁTICA E ESTATÍSTICA (IME-USP). **Introdução à Criptologia (MAC0336/MAC5723): notas de aula**. São Paulo: USP. Disponível em: <https://www.ime.usp.br/~rt/mac57232008/index.html>. Acesso em: 23 jun. 2026.

10 VIDEO DE APRESENTAÇÃO

<https://drive.google.com/file/d/1XEA0j04Fz8Svalr9T8oflLtoTZlcmMe/view>

		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
NOME: Wendel Richard de Carvalho		TURMA: TT8P68		RA: T214417	
CURSO: Sistemas de Informação		CAMPUS:		SEMESTRE: 1º/2º	
CÓDIGO DA ATIVIDADE: IPE — Introdução à Programação Estruturada		SEMESTRE: 2026/1		ANO GRADE: 2026	
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
16/02/2026	Formação do grupo e escolha da técnica criptográfica RSA	3			
02/03/2026	Pesquisa, fundamentação teórica e redação da dissertação sobre RSA	8			
30/03/2026	Desenvolvimento do algoritmo RSA em Python e interface gráfica	10			
20/04/2026	Testes de cifragem/decifragem e redação do relatório de código	7			
04/05/2026	Revisão final, formatação ABNT e gravação do vídeo de apresentação	8			
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.					
TOTAL DE HORAS ATRIBUÍDAS: 36					
AVALIAÇÃO:					
(Aprovado ou Reprovado)					
NOTA:					
DATA: ____ / ____ / ____					

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO					